

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2

Total Marks:20

Annexure A

INTEGRATED

EXPERIMENTS

Code of Conduct:

I ATHAVAN K(192524036)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Athavan K

Annexure A

**INTEGRATED
EXPERIMENTS**

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / PlayTennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, wellstructured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using Python and scikit-learn.

(a) Data Loading, Pre-processing, and Train-Test Split

The Iris dataset (150 samples, 4 numeric features, 3 balanced classes) was loaded using scikit-learn's built-in loader. The target class was label-encoded as 0 = setosa, 1 = versicolor, 2 = virginica. The dataset was checked for missing values (none found), so no imputation was required. Since all features are already numeric, no categorical encoding was needed for the input features. The data was split into 70% training (105 samples) and 30% testing (45 samples) using stratified sampling to preserve class balance.

Python Code

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df['species_name'] = df['species'].map(dict(enumerate(iris.target_names)))

print(df.head())
print(df.dtypes)
print(df.isnull().sum())

X = df[iris.feature_names]
y = df['species']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
```

First 5 rows of the dataset:

sepal length	sepal width	petal length	petal width	species	species_name
5.1	3.5	1.4	0.2	0	setosa
4.9	3.0	1.4	0.2	0	setosa
4.7	3.2	1.3	0.2	0	setosa
4.6	3.1	1.5	0.2	0	setosa
5.0	3.6	1.4	0.2	0	setosa

Data types of each column:

```
sepal length (cm)  float64
sepal width (cm)   float64
petal length (cm)  float64
petal width (cm)   float64
species            int64
species_name       object
```

Missing values per column: 0 (for every column)

Dataset shape: (150, 6) | Training set: 105 rows | Testing set: 45 rows

(b) Building the Decision Tree (Gini Index) and Root Node Justification

```
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree

model = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
model.fit(X_train, y_train)
print(export_text(model, feature_names=list(iris.feature_names)))
```

Tree structure (text form):

```
|--- petal length (cm) <= 2.45
| |--- class: setosa
|--- petal length (cm) > 2.45
| |--- petal width (cm) <= 1.55
| | |--- petal length (cm) <= 4.95 --> class: versicolor
| | |--- petal length (cm) > 4.95 --> class: virginica
| |--- petal width (cm) > 1.55
| | |--- petal width (cm) <= 1.70
| | | |--- sepal width (cm) <= 2.85 --> class: versicolor
| | | |--- sepal width (cm) > 2.85 --> class: virginica
| | |--- petal width (cm) > 1.70
```

			--- petal length (cm) <= 4.85 --> class: versicolor
			--- petal length (cm) > 4.85 --> class: virginica

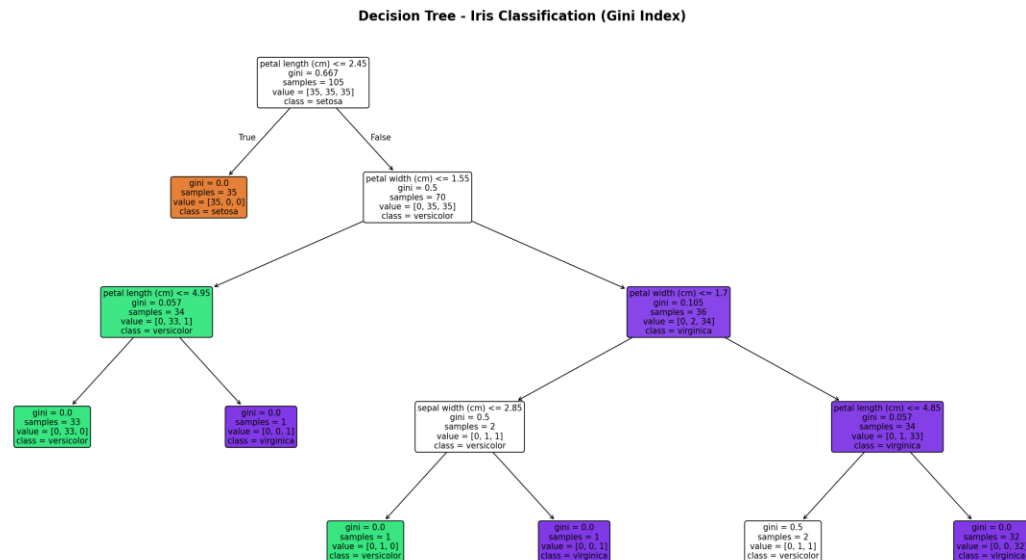


Figure 1: Trained Decision Tree structure (Gini index, max_depth = 4)

Root Node Identification and Justification

The root node splits on 'petal length (cm) <= 2.45'. This feature was chosen automatically by CART because it produces the highest reduction in Gini impurity among all candidate splits at the top of the tree:

- Gini impurity before the split (at the root, all 3 classes mixed): 0.667.
- This single split perfectly isolates all 35 setosa samples into a pure leaf (Gini = 0.0), while sending all versicolor and virginica samples to the other branch.
- Feature importance scores computed by the trained model confirm this: petal length = 0.549 (highest), petal width = 0.437, sepal width = 0.015, sepal length = 0.000.

Because petal length gives the largest, purest, single-feature separation between classes (setosa is linearly separable from the other two species on this feature alone), the algorithm selects it as the first (root) decision, exactly matching the well-known biological fact that setosa has distinctly smaller petals than the other two Iris species.

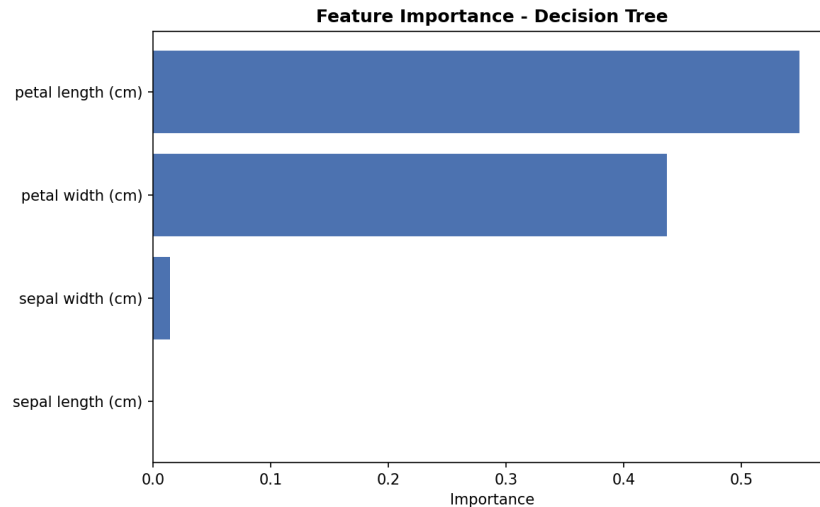


Figure 2: Feature importance scores from the trained Decision Tree

(c) Model Evaluation

```
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

y_pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))
```

Accuracy Score: 0.8889 (88.89%)

Confusion Matrix:

```
[[15  0  0]
 [ 0 12  3]
 [ 0  2 13]]
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.86	0.80	0.83	15
virginica	0.81	0.87	0.84	15
accuracy			0.89	45

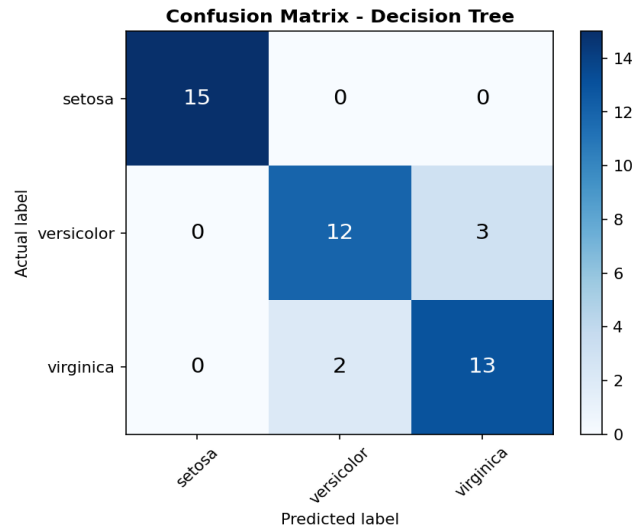


Figure 3: Confusion matrix heatmap on the test set

Misclassified Instances (at least two, as required)

Sample Idx	Features (SL, SW, PL, PW)	Actual	Predicted
56	6.3, 3.3, 4.7, 1.6	versicolor	virginica
85	6.0, 3.4, 4.5, 1.6	versicolor	virginica
77	6.7, 3.0, 5.0, 1.7	versicolor	virginica
138	6.0, 3.0, 4.8, 1.8	virginica	versicolor
106	4.9, 2.5, 4.5, 1.7	virginica	versicolor

Comment on Performance

The model achieves 88.89% overall accuracy on unseen test data. The setosa class is classified with perfect precision and recall (1.00), confirming that it is linearly separable from the other two species using petal length alone, exactly as identified at the root node. All 5 misclassifications occur between versicolor and virginica — the two biologically similar species whose petal length/width ranges overlap (roughly 4.5–5.0 cm petal length and 1.5–1.8 cm petal width for both classes). This is consistent with real-world botanical measurements, where these two species are known to be harder to separate than setosa. The confusion matrix shows the errors are almost symmetric (3 versicolor mislabelled as virginica, 2 virginica mislabelled as versicolor), indicating the decision boundary near petal width ≈ 1.55 –1.7 cm is the primary source of error. Increasing max_depth or using an ensemble (e.g., Random Forest) would likely reduce this boundary overlap error further, at the cost of interpretability.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a 4×4 Grid World environment and observe the agent's learned policy.

(a) Environment Definition, Q-Table Initialization, and Hyperparameters

The environment is a 4×4 grid (16 states, numbered 0–15 row-major). The agent starts at state 0 (top-left) and must reach the goal at state 15 (bottom-right). Three cells (states 5, 7, 11) are defined as obstacles that penalise the agent. The action space has 4 discrete actions: Up, Down, Left, Right. Moving into a wall keeps the agent in the same row/column (no state change beyond grid boundaries).

Element	Definition
States	16 states (4×4 grid), indexed 0–15
Actions	Up, Down, Left, Right
Start state	State 0, position (row 0, col 0)
Goal state	State 15, position (row 3, col 3) → reward = +10, episode ends
Obstacle states	States 5, 7, 11 → reward = −5 (episode continues)
Step reward	−1 for every other move (encourages the shortest path)
Q-table initialization	16 × 4 matrix, initialized to all zeros

Hyperparameters used:

- α (learning rate) = 0.1
- γ (discount factor) = 0.9
- ϵ (exploration rate, ϵ -greedy) = 0.2
- Episodes = 200 (exceeds the minimum of 100 required)
- Max steps per episode = 100 (safety cap to avoid infinite loops)

Python Code – Environment Setup

```
import numpy as np, random
GRID_SIZE = 4
ACTIONS = ['Up','Down','Left','Right']
START_STATE, GOAL_STATE = 0, 15
OBSTACLE_STATES = [5, 7, 11]

def step(state, action):
```

```

r, c = divmod(state, GRID_SIZE)
if action=='Up': r = max(r-1,0)
if action=='Down': r = min(r+1,GRID_SIZE-1)
if action=='Left': c = max(c-1,0)
if action=='Right': c = min(c+1,GRID_SIZE-1)
ns = r*GRID_SIZE + c
if ns==GOAL_STATE: return ns, 10, True
if ns in OBSTACLE_STATES: return ns, -5, False
return ns, -1, False

```

ALPHA, GAMMA, EPSILON, EPISODES = 0.1, 0.9, 0.2, 200
Q = np.zeros((16, 4)) # Q-table initialized with zeros

(b) Running Q-Learning and Q-Table Evolution

```

for ep in range(1, EPISODES+1):
    state = START_STATE
    for t in range(100):
        if random.random() < EPSILON:
            a = random.randint(0,3) # explore
        else:
            a = int(np.argmax(Q[state])) # exploit
        ns, r, done = step(state, ACTIONS[a])
        Q[state,a] += ALPHA*(r + GAMMA*np.max(Q[ns]) - Q[state,a])
        state = ns
    if done: break

```

Q-values were recorded at Episodes 1, 50, and 100 for two representative states:
State 0 (the start state) and State 10 (a mid-grid state adjacent to the goal path).

State 0 — position (row 0, col 0):

Episode	Up	Down	Left	Right
1	-0.199	-0.280	-0.199	-0.199
50	-2.118	-1.864	-2.127	-2.062
100	-1.601	1.222	-1.640	-2.114

State 10 — position (row 2, col 2):

Episode	Up	Down	Left	Right
1	-0.100	-0.100	0.000	0.000
50	0.028	7.437	0.190	-1.314
100	0.587	7.991	0.981	-0.881

Interpretation of Q-table evolution: At episode 1, all Q-values are close to zero or slightly negative — the agent has barely started learning and its estimates come from single, mostly random transitions. By episode 50, values at State 10 already show a strong positive Q-value for 'Down' (7.437), since that action moves directly toward the goal — the effect of reward back-propagation (via the $\gamma \cdot \max Q(\text{next})$ term) reaching backward through the state space after the agent first stumbles onto the goal. By episode 100, this preference has strengthened further (Down = 7.991) and neighbouring actions have also picked up small positive values through generalisation of the Bellman backup, while actions leading toward obstacles (e.g., Right from state 10, heading toward obstacle state 11) remain negative. State 0, being furthest from the goal, converges more slowly, but by episode 100 it already shows 'Down' as the clear best action (1.222 vs negative alternatives), correctly starting the agent toward the goal.

(c) Final Learned Policy, Cumulative Reward, and Convergence

```
policy = {s: ACTIONS[int(np.argmax(Q[s]))] for s in range(16)
          if s != GOAL_STATE and s not in OBSTACLE_STATES}
print(policy)
```

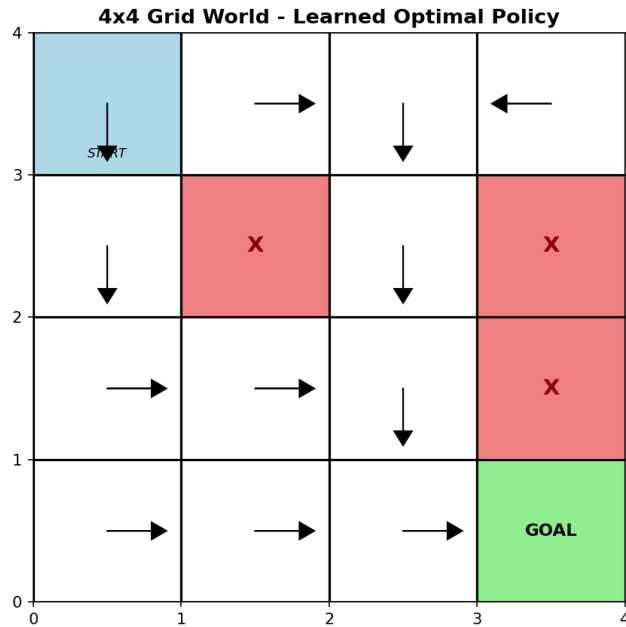


Figure 4: 4×4 Grid World with the final learned optimal policy (arrows show the best action per state)

Final policy grid (row-major, arrows = optimal action, X = obstacle, G = goal):

v	>	v	<
v	X	v	X
>	>	v	X
>	>	>	G

Reading the arrows from the Start state (top-left) confirms the agent has learned a valid obstacle-avoiding path to the goal: Down, Down, Right, Right, Down, Right, Right, Right → reaching state 15 while never entering states 5, 7, or 11.

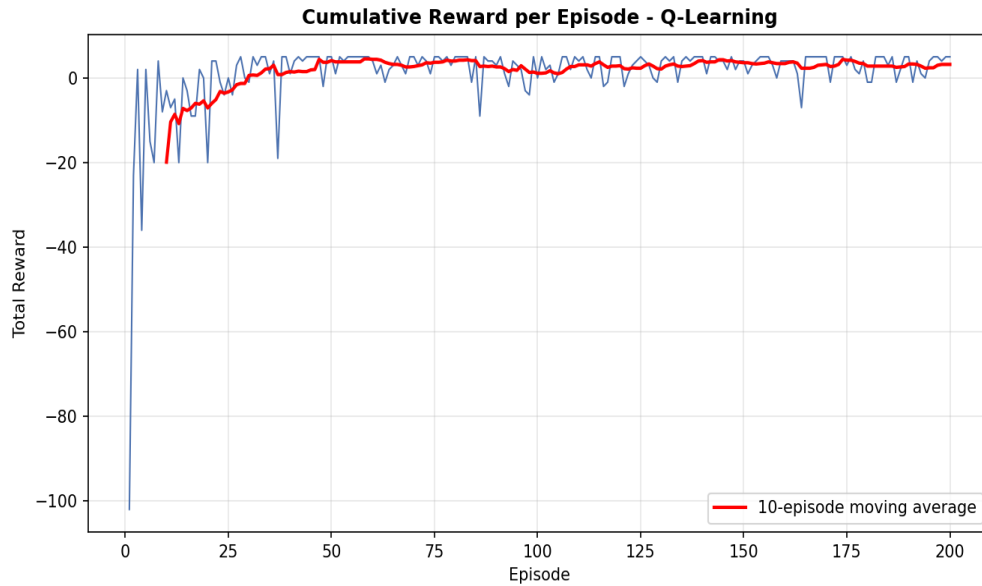


Figure 5: Cumulative reward per episode, with 10-episode moving average

Convergence Behaviour Justification

The reward plot shows the classic Q-learning convergence pattern:

- Early episodes (1–20): highly negative and volatile rewards (as low as -102), because the ϵ -greedy agent is mostly taking random actions and often wanders into obstacles or times out before reaching the goal. Average reward for the first 20 episodes: -13.50 .
- Middle episodes (20–60): rewards rise sharply as the Bellman update propagates the $+10$ goal reward backward through the Q-table, so the agent increasingly exploits the greedy action rather than exploring.
- Later episodes (60–200): rewards stabilise around $+3$ to $+5$ per episode (the theoretical optimum for the shortest path is $+10$ minus one -1 step-penalty per move along a ~ 6 – 7 step path). Average reward for the last 20 episodes: 3.25 , and the moving-average curve flattens, indicating the Q-values have converged and the greedy policy is now near-optimal.

This convergence is expected because the learning rate $\alpha = 0.1$ is small enough to ensure stable incremental updates rather than oscillation, the discount factor $\gamma = 0.9$ sufficiently propagates the terminal reward across the grid, and $\epsilon = 0.2$ keeps a modicum of exploration throughout, which is why some negative dips still occasionally appear even in later episodes (the agent occasionally still explores a random, sub-optimal action).

